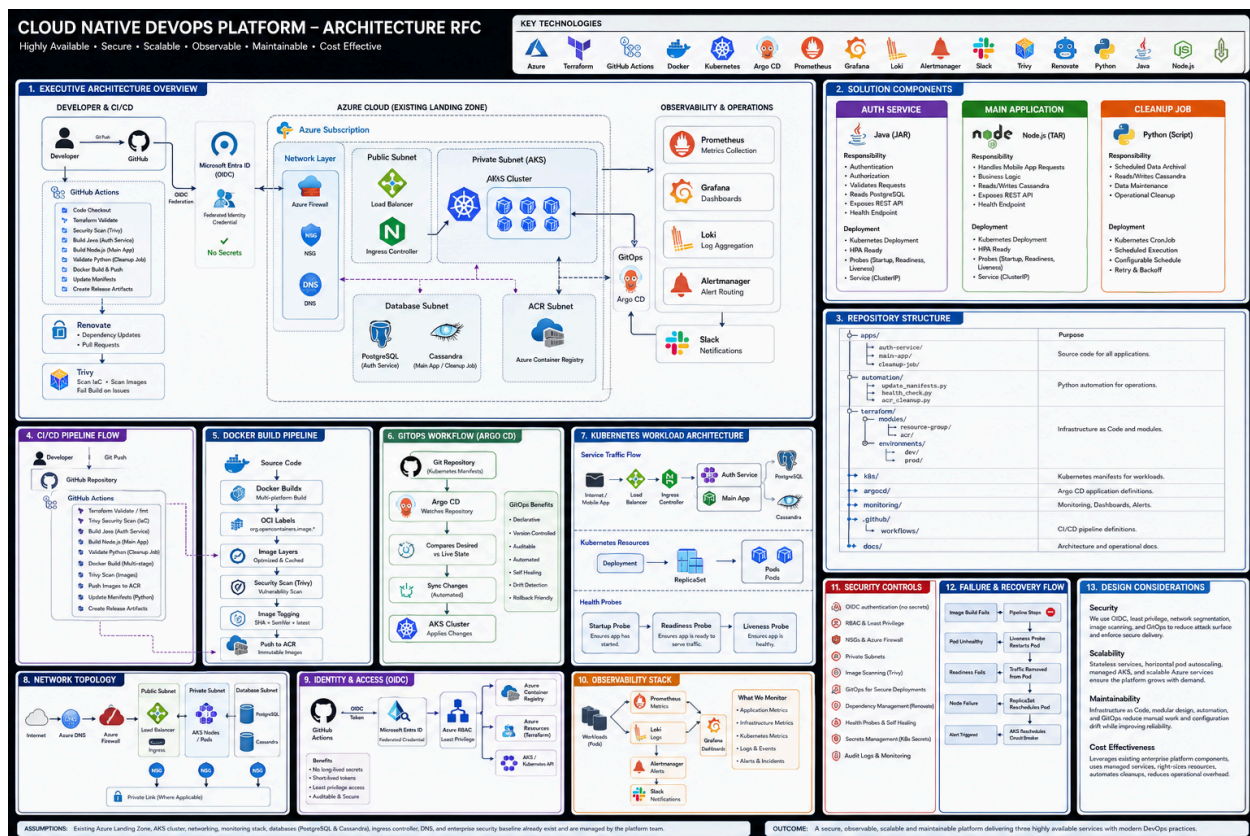


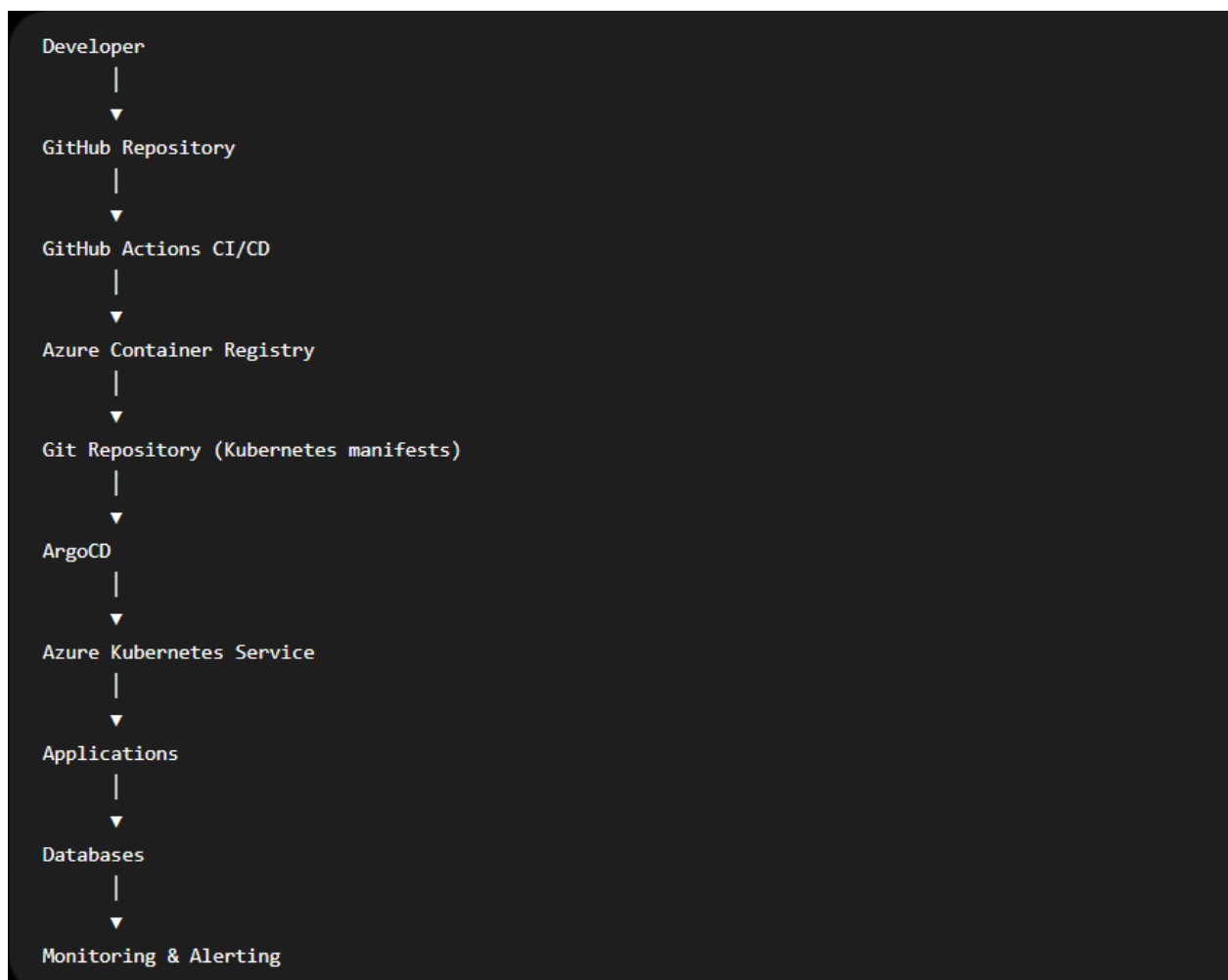
Darktrace Architecture Design

The platform is designed to follow a modern cloud-native workflow where infrastructure provisioning, application delivery and application operations are independent but connected through automation.



Overall Architecture

The overall flow is:



To follow best practices, developers never deploy directly into Kubernetes. Instead, every deployment originates from Git, making Git the single source of truth.

Infrastructure Layer

The infrastructure shown in the diagram represents the Azure platform hosting the applications.

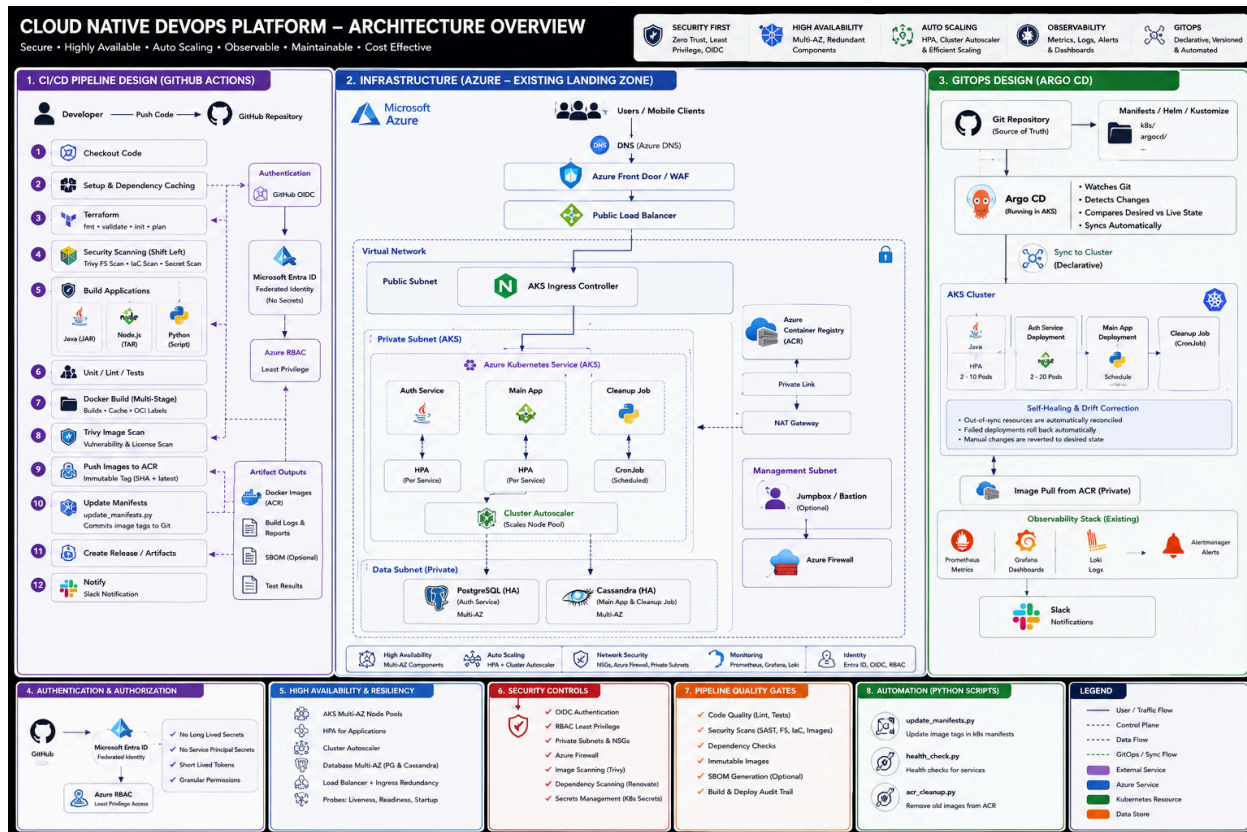
Our Terraform code provisions:

- Azure Resource Group
- Azure Container Registry

while assuming the following enterprise components already exist:

- Azure Subscription
 - Virtual Network
 - AKS Cluster
 - Microsoft Entra ID
 - Azure Load Balancer
 - Azure Firewall
 - Network Security Groups
 - Azure DNS
 - PostgreSQL
 - Cassandra
-

CI/CD and GITOPS



Identity & Authentication

To authenticate from git to the cloud infrastructure **OIDC (OpenID Connect)** will be used.

Instead of storing Azure credentials inside GitHub Secrets, GitHub Actions authenticates directly with Microsoft Entra ID using a federated identity.

The authentication flow is:



This is reflected in your `docker.yml` workflow:

- `azure/login`
- `client-id`
- `tenant-id`
- `subscription-id`

These values identify the federated application, while the authentication itself is performed using short-lived OIDC tokens rather than long-lived secrets.

This improves security by:

- eliminating stored Azure credentials
- reducing credential rotation
- following Microsoft's recommended authentication model

Infrastructure as Code

Terraform provisions Azure resources before applications are deployed.

The repository contains reusable modules:

```
terraform/  
  
modules/  
  resource-group/  
  acr/  
  
environments/  
  dev/
```

Terraform will be sitting before the application deployment process because infrastructure must exist before containers can be published.

Terraform provides:

- **repeatability**
 - **version control**
 - **consistency**
 - **modular infrastructure**
 - **easier maintenance**
-

Continuous Integration

The CI/CD section will directly correspond to the GitHub Actions workflows in the repository.

When code is pushed:

```
Developer  
  
↓  
  
GitHub  
  
↓  
  
GitHub Actions
```

The pipeline will then perform multiple stages.

Validation

- Terraform formatting
- Terraform validation
- Security scanning

These steps ensure infrastructure changes are syntactically correct before deployment.

Application Builds

The workflow builds all three applications independently.

Auth Service

Java that Produces:

```
JAR
```

Main Application

Node.js

Produces:

```
Node application
```

Cleanup Job

Python

Validates

```
cleanup.py
```

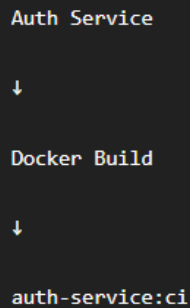
before packaging.

Docker Build

After the applications are built, Docker images will be created.

This is showcased in the Build job in [docker.yml](#).

Each application is containerised separately.



```
Auth Service
↓
Docker Build
↓
auth-service:ci
```

A diagram illustrating the build process for the Auth Service. It shows a vertical flow starting with 'Auth Service', followed by a downward arrow, then 'Docker Build', another downward arrow, and finally the resulting Docker image 'auth-service:ci'.

The same occurs for:

- main-app
- cleanup-job

Images will be exported as artifacts before the publish stage.

Security Scanning

The architecture will include Trivy before publication.

Its purpose will be to scan:

- container images
- operating system packages
- dependencies
- infrastructure definitions

before images are published.

If vulnerabilities are detected above an acceptable threshold, the pipeline will fail before deployment.

This is of a Shift Left security approach where issues are detected during CI rather than in production.

Publish Stage

Only the main branch will be allowed to publish.

This matches GitHub workflow:

```
</> YAML  
  
if: github.ref == 'refs/heads/main'
```

Images are then pushed into Azure Container Registry.

Each image will receive:

```
latest  
  
and  
  
Git SHA
```

The SHA tag will provides immutable versioning, allowing ArgoCD to deploy specific image versions reliably.

Azure Container Registry

ACR acts as the central image repository.

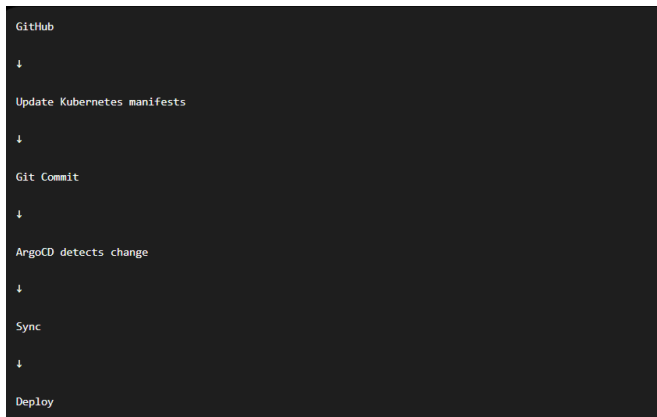
Rather than building images inside Kubernetes, Kubernetes simply pulls pre-built images from ACR.

This separates build responsibilities from runtime responsibilities.

GitOps

Instead of deploying directly from GitHub Actions into Kubernetes, the architecture uses GitOps.

The flow will be:



This is why the repository contains:

```
argocd/  
  
k8s/
```

GitHub Actions will prepare the deployment artefacts, while ArgoCD will be responsible for applying them to the cluster.

This provides:

- deployment history
 - rollback capability
 - automatic reconciliation
 - reduced configuration drift
-

Kubernetes

Within AKS, three workloads will exist.

Auth Service

Runs as a Deployment.

Includes:

- ReplicaSet (3 for high availability)
- Pods
- Service

Health probes include:

- Startup Probe
- Readiness Probe
- Liveness Probe

The readiness probe prevents traffic reaching pods before they are ready.

The liveness probe automatically restarts unhealthy containers.

The startup probe allows slower application startup without premature restarts.

Main Application

Runs as another Deployment.

Like the Auth Service, it will includes:

- ReplicaSets (3 for high availability)
- Services
- Health Probes

Traffic reaches this application only after successful authentication by the Auth Service.

Cleanup Job

Rather than a Deployment, this runs as a Kubernetes CronJob.

It periodically:

- archives historical data
- removes stale records
- performs housekeeping against Cassandra

Because it is scheduled, resources are only consumed when required.

High Availability

- High availability will be achieved through several mechanisms:
 - Multiple pod replicas ensure application availability.
 - ReplicaSets automatically recreate failed pods.
 - Kubernetes Services distribute traffic across healthy pods.
 - Readiness probes remove unhealthy pods from service.
 - Liveness probes restart failed containers.
 - The Azure Load Balancer distributes incoming traffic to the cluster.
 - This combination provides resilience against individual pod failures.
-

Autoscaling

Although not fully implemented in the repository, the architecture assumes Kubernetes supports Horizontal Pod Autoscaling (HPA).

This will also use Prometheus or Kubernetes metrics to increase or decrease replicas based on CPU, memory or custom application metrics.

Data Layer

Two separate databases will exist.

PostgreSQL

Used exclusively by the Auth Service.

Authentication requests query PostgreSQL before allowing downstream requests.

Keeping authentication data isolated improves separation of concerns.

Cassandra

Used by:

- Main Application
- Cleanup Job

The Main Application performs application reads and writes.

The Cleanup Job archives or removes historical records.

Sharing Cassandra ensures operational consistency while separating application logic from maintenance tasks.

Observability

One of the most vital aspects of the design is the dedicated observability stack.

Each application exposes metrics.

Prometheus scrapes those metrics.

Grafana visualises them.

Loki centralises application logs.

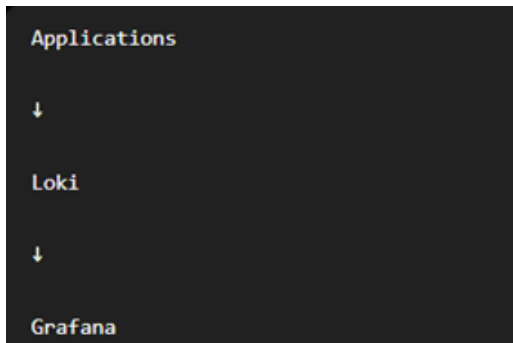
Alertmanager evaluates alert rules.

Slack receives operational notifications.

The operational flow is:



while logs will flow through:



This will provide engineers with metrics, logs and alerts from a single operational platform.

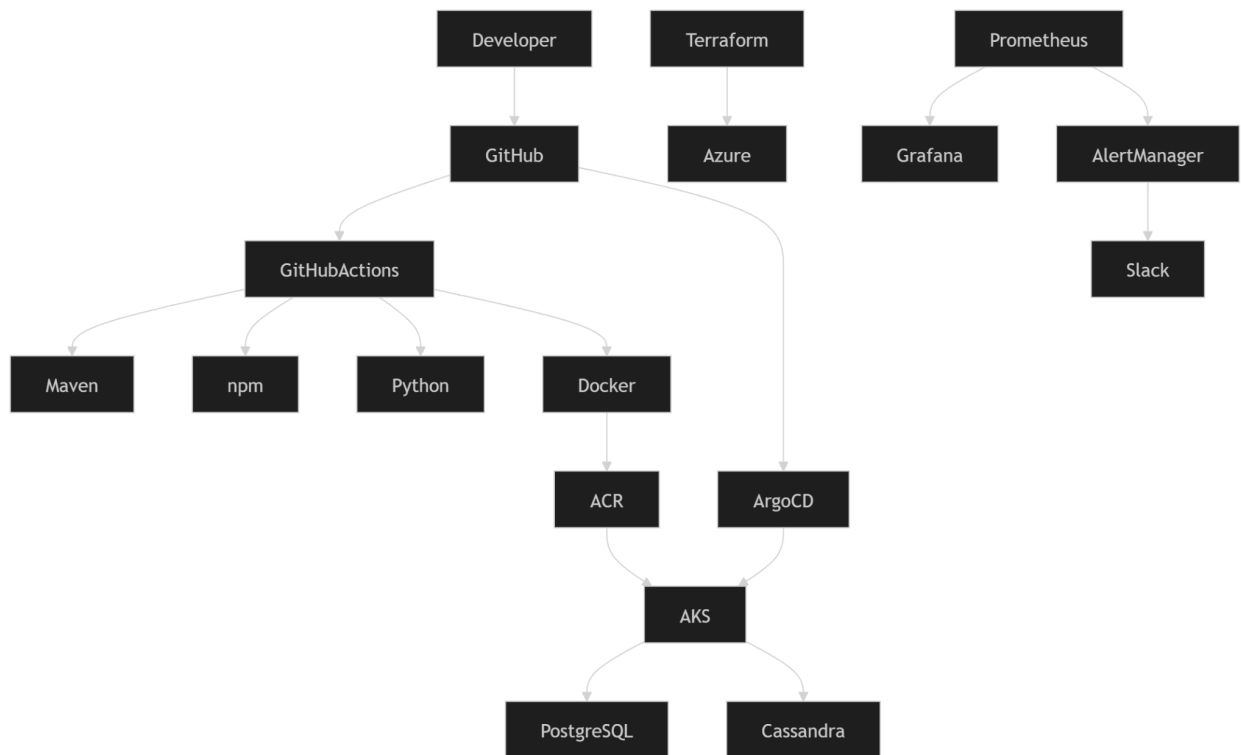
Automation

The Python automation scripts will support the operational workflow around deployments.

- **update_manifests.py** updates Kubernetes manifests with newly built image tags so ArgoCD deploys the correct version.
- **health_check.py** provides automated health verification that can be incorporated into CI/CD or operational checks.
- **acr_cleanup.py** removes stale or unused container images from Azure Container Registry, helping control storage growth and keeping the registry tidy.

These scripts will complement the GitHub Actions workflows by automating repetitive operational tasks rather than relying on manual intervention.

End-to-End Flow



•

A developer commits code to GitHub.

- GitHub Actions validates Terraform, builds the Java, Node.js and Python applications, scans them with Trivy, and creates Docker images.
- Images are published to Azure Container Registry using secure OIDC authentication.
- Kubernetes manifests are updated to reference the new immutable image tags.
- ArgoCD detects the manifest changes and synchronises the desired state to the existing AKS cluster.
- AKS performs a rolling deployment using Deployments, ReplicaSets and health probes to maintain application availability.
- The Auth Service authenticates inbound requests against PostgreSQL before forwarding authorised traffic to the Main Application.
- The Main Application processes mobile client requests using Cassandra as its primary datastore, while the Cleanup Job periodically archives older data.
- Throughout runtime, Prometheus collects metrics, Loki aggregates logs, Grafana provides dashboards, and Alertmanager sends notifications to Slack, giving operations teams continuous visibility into the health and performance of the platform.

Estimated Cost Breakdown (based off azure cost charts)

The solution will be designed to integrate with an existing Azure enterprise platform, with only the resources required by the application being provisioned. Estimated monthly costs are shown below and will vary depending on workload, region and utilisation.

Azure Service	Purpose	Estimated Monthly Cost
Azure Kubernetes Service (AKS)	Container orchestration	~£50 (control plane) + worker node costs
AKS Worker Nodes	Run application workloads	~£300–£500
Azure Container Registry (ACR)	Container image storage	~£5 (Basic) / ~£30 (Standard)
Azure Load Balancer	External traffic distribution	~£20
Azure Firewall	Network security (assumed existing)	~£900
PostgreSQL Flexible Server	Authentication database	~£200–£300
Cassandra Cluster	Application database	~£500–£700
Azure Monitor / Log Analytics	Platform monitoring	Usage-based

Cost Optimisation Measures

The architecture incorporates several measures to minimise operational costs while maintaining security and availability:

- **Right-sized compute** – Kubernetes workloads are intended to scale only to the resources required, avoiding unnecessary CPU and memory allocation.
- **Horizontal Pod Autoscaling (assumed)** – Additional application replicas are created only when demand increases and reduced during quieter periods.
- **Basic Azure Container Registry** – A Basic SKU is sufficient for the exercise, with the option to upgrade as requirements grow.
- **Shared enterprise infrastructure** – Existing networking, identity, monitoring and Kubernetes services are assumed, avoiding duplication of platform resources.
- **Infrastructure as Code** – Terraform ensures only required resources are provisioned and simplifies lifecycle management.

- **Automated container cleanup** – Python automation removes stale images from Azure Container Registry to reduce storage consumption.
- **GitOps deployments** – Automated deployments reduce operational effort and minimise manual intervention.
- **Open-source tooling** – Terraform, ArgoCD, Prometheus, Grafana OSS, Loki, Alertmanager, Trivy, Renovate and Python automation are all open source, reducing licensing costs while providing enterprise-grade capabilities.
- **Containerised microservices** – Running applications as containers on a shared AKS cluster improves resource utilisation compared to dedicated virtual machines.

Overall, the design balances **cost efficiency**, **operational simplicity**, and **scalability** by leveraging Azure managed services where appropriate and open-source technologies for deployment, monitoring, security scanning and automation.